



Universidad Autónoma de Baja California
Facultad de Ingeniería Mexicali
Ingeniería en Computación

Reporte de Práctica 07
Llamadas al Sistema en MASMx86

Organización y Arquitectura de Computadoras
Ing. Félix Arredondo Santamaría

Moya Monreal Erick Anselmo — 1110604

Fecha de entrega
01/MAYO/2026

Mexicali, Baja California
29 de mayo de 2026

Índice

1. Introducción	2
2. Problemática y Justificación	2
3. Objetivos de la práctica	3
4. Marco Teórico	3
4.1. Lenguaje Ensamblador y la API de Windows	3
4.2. Procedimientos y la Convención <code>stdcall</code>	4
4.3. Paso de Parámetros por la Pila	4
4.4. Cadenas Terminadas en Nulo y Cálculo de Longitud	4
4.5. Lectura de Consola y Limpieza del Buffer	4
4.6. Conversión entre Cadena y Entero	5
5. Procedimiento	5
6. Resultados con pruebas	6
6.1. Actividad 1: <code>ImprimirCadena</code>	6
6.2. Actividad 2: <code>LeerCadena</code>	8
6.3. Actividad 3: <code>LeerNumero</code> e <code>ImprimirNumero</code>	10
6.4. Resumen de las funciones implementadas	14
7. Conclusiones	15
Referencias	16
A. Repositorio de código	16

1. Introducción

El lenguaje ensamblador concede al programador un control directo sobre los registros del procesador, la pila y las llamadas al sistema operativo, pero a cambio le exige construir manualmente abstracciones que en un lenguaje de alto nivel vienen resueltas de fábrica. Operaciones tan cotidianas como imprimir una cadena, leer texto del teclado o convertir un número a su representación decimal no existen como instrucciones del procesador, deben programarse a partir de las funciones que ofrece el sistema operativo y de la manipulación byte a byte de la memoria.

En esta práctica se desarrolló un conjunto de funciones reutilizables en **MASM x86** que encapsulan las operaciones básicas de entrada y salida por consola apoyándose en la API de Windows. Se implementan cuatro procedimientos: **ImprimirCadena**, que muestra una cadena de cualquier longitud calculando su tamaño automáticamente; **LeerCadena**, que captura texto del usuario y lo deja limpio y terminado en nulo; y la pareja **LeerNumero** e **ImprimirNumero**, que traducen entre la representación textual de un número y su valor entero de 32 bits con signo.

2. Problemática y Justificación

A diferencia de un lenguaje de alto nivel, donde una sola instrucción como **printf** o **scanf** resuelve la comunicación con la consola, el ensamblador no dispone de funciones de entrada y salida integradas. Toda interacción con el usuario debe canalizarse a través de las funciones que expone el sistema operativo, y para ello es indispensable obtener primero los descriptores (*handles*) de los dispositivos estándar de entrada y salida, preparar la pila con los argumentos en el orden correcto y respetar la convención de llamada que el sistema espera [1].

Esto plantea varios retos concretos. La función **WriteConsoleA** necesita conocer de antemano cuántos caracteres debe imprimir, dato que no se conoce si las cadenas se almacenan terminadas en nulo al estilo de C; por ello hay que recorrerlas carácter por carácter para medir su longitud. La lectura con **ReadConsoleA** introduce al final del buffer los caracteres de retorno de carro y salto de línea (0Dh, 0Ah) que genera la tecla *Enter*, los cuales deben eliminarse para que la cadena sea reutilizable. Finalmente, la consola solo maneja texto, de modo que un número entero debe convertirse dígito por dígito a caracteres ASCII para mostrarse, y a la inversa para leerse [2].

3. Objetivos de la práctica

Objetivo General

Diseñar e implementar en lenguaje ensamblador MASM x86 un conjunto de funciones reutilizables para la entrada y salida de cadenas y números enteros por consola, empleando la API de Windows y la convención de llamada `stdcall`, de modo que cada función opere sobre datos de longitud arbitraria sin necesidad de especificarla manualmente.

Objetivos Específicos

- **Obtener y emplear** los *handles* estándar de entrada y salida mediante `GetStdHandle` para invocar correctamente a `WriteConsoleA` y `ReadConsoleA`.
- **Implementar** la función `ImprimirCadena`, capaz de calcular automáticamente la longitud de una cadena terminada en nulo recorriéndola carácter por carácter hasta el terminador.
- **Desarrollar** la función `LeerCadena` para capturar texto del usuario de forma segura, eliminando los caracteres `0Dh` y `0Ah` que añade la tecla *Enter* y dejando la cadena terminada en nulo.
- **Construir** las funciones `LeerNumero` e `ImprimirNumero` para convertir entre la representación textual y el valor entero de 32 bits con signo, soportando valores negativos y todo el rango válido.
- **Aplicar** la convención de llamada `stdcall`, pasando los parámetros por la pila y empleando procedimientos (`PROC/ENDP`) con preservación de los registros utilizados.

4. Marco Teórico

4.1. Lenguaje Ensamblador y la API de Windows

El lenguaje ensamblador mantiene una correspondencia casi uno a uno con el lenguaje máquina de una arquitectura específica, exponiendo los registros, los modos de direccionamiento y la organización de la memoria [1]. Como el procesador por sí solo no sabe dialogar con la pantalla o el teclado, los programas de consola en Windows delegan esa tarea en la *Application Programming Interface* (API) del sistema operativo. Las funciones `GetStdHandle`, `WriteConsoleA` y `ReadConsoleA`, contenidas en `kernel32.lib`, constituyen la interfaz mínima para escribir y leer en la consola [3].

4.2. Procedimientos y la Convención `stdcall`

Un procedimiento (`PROC...ENDP`) es el equivalente en ensamblador a una función, un bloque de código reutilizable al que se transfiere el control mediante `CALL` y del que se regresa con `RET`. Para que llamante y llamado se entiendan deben acordar una *convención de llamada* [4]. El programa se declara con `.model flat, stdcall`, por lo que tanto el código propio como las funciones de la API siguen la convención `stdcall`, en la cual los argumentos se apilan de derecha a izquierda y es la **propia función llamada** quien libera la pila antes de retornar (`RET n`). Esto contrasta con la convención `cdecl`, donde la limpieza recae en el llamante. La directiva `PROTO` declara la firma de cada función para que el ensamblador verifique el número de argumentos.

4.3. Paso de Parámetros por la Pila

Bajo `stdcall`, antes de un `CALL` los argumentos se colocan en la pila con `PUSH`. Dentro de un `PROC` declarado con parámetros con nombre (por ejemplo `pBuffer:DWORD`), el ensamblador genera de manera transparente el prólogo y el epílogo que crean el marco de pila, de modo que cada parámetro puede leerse por su nombre como si fuera una variable local [2]. Esta característica permite escribir funciones legibles sin calcular manualmente los desplazamientos sobre `EBP`. Sin embargo, al usar la pila para pasar parámetros es crucial respetar el orden de apilado y asegurarse de que cada función limpie la pila según lo estipulado por la convención, para evitar corrupciones de memoria o retornos a direcciones erróneas.

4.4. Cadenas Terminadas en Nulo y Cálculo de Longitud

Siguiendo la convención del lenguaje C, las cadenas se almacenan como secuencias de bytes terminadas en un carácter nulo (0). Esta representación tiene una ventaja, no es necesario guardar la longitud por separado, pero obliga a recorrer la cadena byte a byte para conocer su tamaño [1]. El patrón empleado consiste en avanzar un puntero e incrementar un contador mientras el byte apuntado sea distinto de cero, deteniéndose en el terminador. Ese contador es justamente el número de caracteres que `WriteConsoleA` requiere como parámetro, lo que permite imprimir cadenas de cualquier longitud sin declararla de antemano.

4.5. Lectura de Consola y Limpieza del Buffer

`ReadConsoleA` deposita en un buffer los caracteres tecleados por el usuario y devuelve, a través de un parámetro de salida, el número de caracteres leídos. Cuando el usuario presiona *Enter*, el sistema añade la secuencia de fin de línea de Windows: retorno de carro (0Dh, `\r`) seguido de salto de línea (0Ah, `\n`) [3]. Si esos dos bytes no se eliminan, la

cadena resultante arrastra saltos de línea espurios que estropean su posterior impresión o comparación. Por ello **LeerCadena** localiza el final de lo leído y sustituye dichos caracteres por el terminador nulo, dejando una cadena limpia y reutilizable.

4.6. Conversión entre Cadena y Entero

La consola únicamente transporta texto, así que un número debe convertirse entre dos representaciones. Para **leer** un número (**LeerNumero**) se aplica el esquema de Horner, partiendo de un acumulador en cero, por cada dígito se multiplica el acumulador por diez y se le suma el valor del dígito, obtenido al restar 48 (el código ASCII del carácter '0') al carácter leído [5]. Para **imprimir** un número (**ImprimirNumero**) se realiza el proceso inverso, se divide repetidamente entre diez y cada residuo, convertido a carácter sumándole 48, se apila; como los dígitos se obtienen del menos al más significativo, la pila (estructura LIFO) los devuelve en el orden correcto al desapilarlos. En ambos casos se gestiona aparte el signo para soportar números negativos en todo el rango de 32 bits.

5. Procedimiento

El desarrollo se organizó en tres actividades incrementales, cada una construyendo una función de entrada/salida más completa que la anterior y reutilizando los conceptos previos. Las actividades realizadas fueron las siguientes:

1. **Actividad 1 — ImprimirCadena.** Se programó un procedimiento que recibe la dirección de una cadena, recorre sus bytes hasta el terminador nulo para medir su longitud y la imprime con **WriteConsoleA**. Se probó con tres cadenas de distinta longitud.
2. **Actividad 2 — LeerCadena.** Se implementó la captura de texto con **ReadConsoleA** sobre un buffer seguro, eliminando los caracteres 0Dh y 0Ah de la tecla *Enter* y terminando la cadena en nulo. Se verificó reimprimiendo el texto capturado.
3. **Actividad 3 — LeerNumero e ImprimirNumero.** Se codificó la conversión de cadena a entero (con manejo de signo) y la conversión inversa de entero a cadena mediante divisiones sucesivas y uso de la pila, probando valores positivos y negativos.

6. Resultados con pruebas

6.1. Actividad 1: ImprimirCadena

El primer procedimiento resuelve el problema de imprimir una cadena sin conocer de antemano su longitud. Recibe por la pila la dirección de inicio de la cadena, la recorre byte a byte incrementando un contador hasta encontrar el terminador nulo y, con esa longitud, invoca a WriteConsoleA. El Código 1 muestra el procedimiento junto con las llamadas desde main.

```
1 main PROC
2     PUSH  STDOUT_HANDLE
3     CALL  GetStdHandle          ; obtener handle de salida
4     MOV   hStdOut,  EAX
5
6     PUSH  OFFSET mensaje1
7     CALL  ImprimirCadena
8     PUSH  OFFSET mensaje2
9     CALL  ImprimirCadena
10    PUSH  OFFSET mensaje3
11    CALL  ImprimirCadena
12    RET
13 main ENDP
14
15 ImprimirCadena PROC STDCALL, pCadena:DWORD
16     push  ecx
17     push  edx
18     mov   edx, pCadena          ; EDX -> inicio de la cadena
19     mov   ecx, 0                ; ECX = contador de longitud
20 _loop_calcular_longitud:
21     cmp   byte ptr [edx], 0     ; byte actual == terminador nulo?
22     je    _fin_loop
23     inc   ecx                  ; un caracter mas
24     inc   edx                  ; avanzar al siguiente byte
25     jmp   _loop_calcular_longitud
26 _fin_loop:
27     push  0
28     push  offset caracteresEscritos
29     push  ecx                  ; longitud calculada
30     push  pCadena              ; direccion de la cadena
31     push  hStdOut
32     call  WriteConsoleA
33     pop   edx
34     pop   ecx
35     RET
36 ImprimirCadena ENDP
```

Listing 1: Procedimiento ImprimirCadena e invocación con tres cadenas.

Análisis de instrucción

Instrucción

```
1      cmp byte ptr [edx], 0
2      je _fin_loop
```

Descripción Compara el byte apuntado por EDX contra el terminador nulo y detiene el conteo cuando lo encuentra.

El operador `byte ptr` indica que el acceso a memoria `[edx]` es de un solo byte, evitando la ambigüedad de tamaño. Mientras el carácter sea distinto de cero, el bucle incrementa el contador ECX y avanza el puntero EDX; al hallar el nulo, JE (*Jump if Equal*) rompe el ciclo. Este recorrido es el que sustituye a la función `strlen` del lenguaje de alto nivel y permite imprimir cadenas de longitud arbitraria.

Análisis de instrucción

Instrucción

```
1      push ecx
2      push pCadena
3      push hStdOut
4      call WriteConsoleA
```

Descripción Apila el número de caracteres, la dirección de la cadena y el *handle* de salida, y llama a la API para imprimir.

Los parámetros de `WriteConsoleA` se colocan en la pila de derecha a izquierda según su firma: el reservado en cero, el puntero a la variable que recibirá los caracteres escritos, el número de caracteres a escribir (ECX, calculado previamente), la dirección del texto y, en la cima, el *handle* de salida obtenido con `GetStdHandle`. Como la función pertenece a la API y sigue `stdcall`, ella misma libera la pila al retornar.

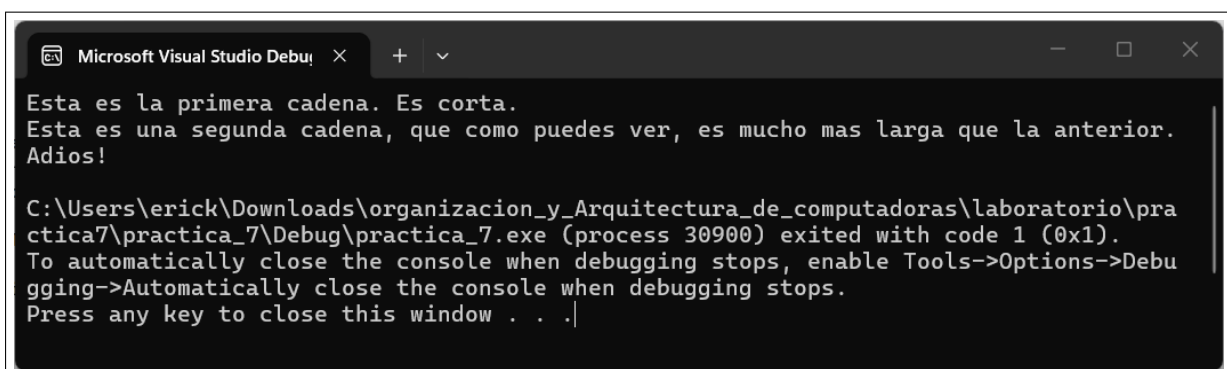


Figura 1: Impresión de tres cadenas de distinta longitud mediante `ImprimirCadena`.

6.2. Actividad 2: LeerCadena

La segunda actividad captura texto del usuario mediante `ReadConsoleA` y lo deja listo para ser reutilizado. El reto principal es eliminar la secuencia `0Dh 0Ah` que la tecla *Enter* agrega al final del buffer. El procedimiento detecta esos dos bytes y los reemplaza por el terminador nulo; si por alguna razón no aparecen, coloca el nulo justo después del último carácter leído. El Código 2 presenta la rutina.

```
1 LeerCadena PROC STDCALL , pBuffer:DWORD
2     push eax
3     push ecx
4     push edx
5
6     push 0
7     push offset leidos           ; recibe el numero de caracteres
8     leidos
9     push 130                     ; maximo a leer
10    push pBuffer                 ; buffer destino
11    push hStdIn
12    call ReadConsoleA
13
14    mov eax, leidos
15    cmp eax, 2                   ; entrada demasiado corta?
16    jnl fin_funcion
17
18    mov edx, pBuffer
19    add edx, eax
20    sub edx, 2                   ; EDX -> penultimo byte leído
21
22    cmp byte ptr [edx], 0Dh      ; retorno de carro?
23    jne no_crlf
24    cmp byte ptr [edx+1], 0Ah    ; salto de linea?
25    jne no_crlf
26    mov byte ptr [edx], 0        ; reemplazar CR por terminador nulo
27    jmp fin_funcion
28
29    no_crlf:
30    mov edx, pBuffer
31    add edx, eax
32    mov byte ptr [edx], 0        ; terminar en nulo al final
33
34    fin_funcion:
35    pop edx
36    pop ecx
37    pop eax
38    ret
39
40 LeerCadena ENDP
```

Listing 2: Procedimiento LeerCadena: captura y limpieza del salto de línea.

Análisis de instrucción

Instrucción

```
1  mov edx, pBuffer
2  add edx, eax
3  sub edx, 2
```

Descripción Calcula la dirección de los dos últimos caracteres leídos para inspeccionar el fin de línea.

Partiendo de la dirección base del buffer se le suma el número de caracteres leídos (**EAX**) para situarse al final, y luego se restan dos posiciones para apuntar al primer byte de la posible secuencia **0Dh 0Ah**. Este cálculo de puntero es indispensable porque la posición del salto de línea depende de cuánto haya escrito el usuario, dato que solo se conoce en tiempo de ejecución gracias al valor que **ReadConsoleA** devuelve en **leídos**.

Análisis de instrucción

Instrucción

```
1  cmp byte ptr [edx], 0Dh
2  jne no_crlf
3  cmp byte ptr [edx+1], 0Ah
4  jne no_crlf
5  mov byte ptr [edx], 0
```

Descripción Verifica que los dos últimos bytes sean **CR** y **LF**, y de ser así los anula con el terminador nulo.

Solo cuando ambas comparaciones confirman la presencia de retorno de carro y salto de línea se sobrescribe el primero de ellos con 0, lo que trunca efectivamente la cadena justo antes del salto de línea. De este modo la cadena queda limpia y terminada en nulo, en condiciones de ser pasada a **ImprimirCadena** u otra función que espere ese formato.

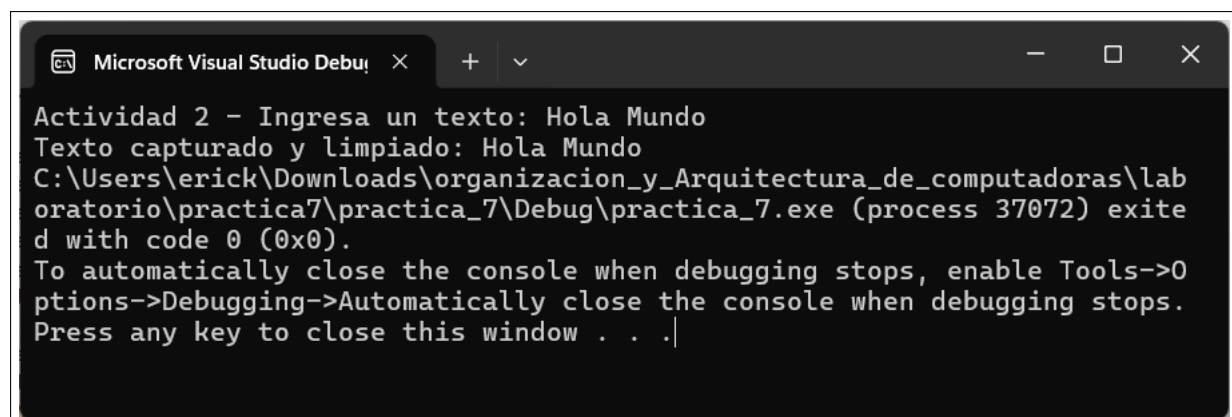


Figura 2: Captura de texto con **LeerCadena** y su posterior impresión ya limpio.

6.3. Actividad 3: LeerNumero e ImprimirNumero

La tercera actividad aborda la conversión entre texto y números enteros de 32 bits con signo. LeerNumero transforma la cadena tecleada en un valor entero que devuelve en EAX, mientras que ImprimirNumero recibe un entero en EAX y lo muestra en consola dígito a dígito. El Código 3 muestra la conversión de cadena a entero.

```
1 LeerNumero PROC STDCALL
2     push ecx
3     push edx
4     push ebx
5     push esi
6
7     push 0
8     push offset rwCount
9     push 30
10    push offset inputBuffer
11    push hStdIn
12    call ReadConsoleA
13
14    mov ecx, rwCount
15    cmp ecx, 2
16    jl _fin_lectura
17    mov byte ptr [inputBuffer + ecx - 2], 0    ; quitar CR/LF
18
19    xor eax, eax    ; acumulador = 0
20    xor ecx, ecx    ; indice = 0
21    mov ebx, 10     ; base decimal
22
23    mov esi, 1      ; signo = +1
24    cmp byte ptr [inputBuffer], '-'
25    jne _ciclo
26    mov esi, -1     ; signo = -1
27    inc ecx         ; saltar el signo
28 _ciclo:
29    movzx edx, byte ptr [inputBuffer + ecx]
30    cmp dl, 0
31    je _fin_conversion
32    sub dl, 48      ; ASCII -> valor del digito
33    push edx
34    mul ebx         ; acumulador *= 10
35    pop edx
36    add eax, edx    ; acumulador += digito
37    inc ecx
38    jmp _ciclo
39 _fin_conversion:
40    imul eax, esi   ; aplicar el signo
41 _fin_lectura:
42    pop esi
```

```
43     pop ebx
44     pop edx
45     pop ecx
46     ret
47 LeerNumero ENDP
```

Listing 3: Procedimiento LeerNumero: conversión de cadena a entero con signo.

Análisis de instrucción

Instrucción

```
1     sub dl, 48
2     mul ebx
3     add eax, edx
```

Descripción Convierte el carácter a su valor numérico y lo integra al acumulador mediante el esquema de Horner.

Al restar 48 (código ASCII de '0') al carácter se obtiene el valor del dígito. Antes de sumarlo se multiplica el acumulador EAX por la base 10 con MUL EBX, desplazando los dígitos previos una posición decimal; luego ADD EAX, EDX incorpora el nuevo dígito. El dígito se preserva temporalmente con PUSH/POP porque MUL sobrescribe EDX con la parte alta del producto. Repetir este paso por cada carácter reconstruye el número completo sin importar cuántas cifras tenga.

Análisis de instrucción

Instrucción

```
1     mov esi, 1
2     cmp byte ptr [inputBuffer], '-'
3     jne _ciclo
4     mov esi, -1
5     inc ecx
```

Descripción Detecta el signo negativo inicial, guarda el signo en ESI y avanza el índice para no procesar el carácter '-' como dígito.

El signo se trata por separado del valor absoluto. Si el primer carácter es un guion, ESI pasa a valer -1 y el índice ECX se incrementa para saltar ese carácter; en caso contrario ESI permanece en +1. Al terminar la conversión, IMUL EAX, ESI aplica el signo al resultado, lo que permite soportar valores negativos como -999 o -2147483648 dentro del rango de 32 bits.

La conversión inversa, de entero a texto, la realiza ImprimirNumero, mostrada en el Código 4. Si el número es negativo imprime primero el signo y lo convierte a positivo; después extrae los dígitos por divisiones sucesivas entre diez, apilándolos para invertir su orden.

```
1 ImprimirNumero PROC STDCALL, valor:SDWORD
```

```
2     push eax
3     push ebx
4     push ecx
5     push edx
6
7     mov eax, valor
8     cmp eax, 0
9     jge _es_positivo
10    push eax                ; preservar el valor negativo
11    mov byte ptr [bufferTemp], '-'
12    push 0
13    push offset rwCount
14    push 1
15    push offset bufferTemp
16    push hStdOut
17    call WriteConsoleA      ; imprimir el signo '-'
18    pop eax
19    neg eax                ; trabajar con el valor absoluto
20    _es_positivo:
21    mov ebx, 10
22    xor ecx, ecx           ; contador de digitos
23    _loop_dividir:
24    xor edx, edx
25    div ebx                ; EAX / 10, residuo en EDX
26    add dl, 48             ; digito -> ASCII
27    push dx                ; apilar el digito
28    inc ecx
29    test eax, eax
30    jnz _loop_dividir
31    _loop_imprimir:
32    pop dx                 ; desapilar en orden inverso
33    mov byte ptr [bufferTemp], dl
34    push ecx
35    push 0
36    push offset rwCount
37    push 1
38    push offset bufferTemp
39    push hStdOut
40    call WriteConsoleA
41    pop ecx
42    dec ecx
43    cmp ecx, 0
44    jg _loop_imprimir
45
46    pop edx
47    pop ecx
48    pop ebx
49    pop eax
50    ret
```

51 `ImprimirNumero ENDP`

Listing 4: Procedimiento `ImprimirNumero`: conversión de entero a cadena con uso de la pila.

Análisis de instrucción

Instrucción

```
1  xor edx, edx
2  div ebx
3  add dl, 48
4  push dx
```

Descripción Extrae el dígito menos significativo como residuo de la división entre diez, lo convierte a carácter y lo apila.

Antes de cada `DIV` se limpia `EDX` con `XOR EDX, EDX`, ya que la división de 32 bits usa el par `EDX:EAX` como dividendo y un residuo basura corrompería el resultado. Tras `DIV EBX`, el cociente queda en `EAX` y el residuo (el dígito) en `EDX`; se le suma 48 para obtener su carácter ASCII y se apila. Como los dígitos salen del menos al más significativo, la pila los almacena en orden inverso al de impresión.

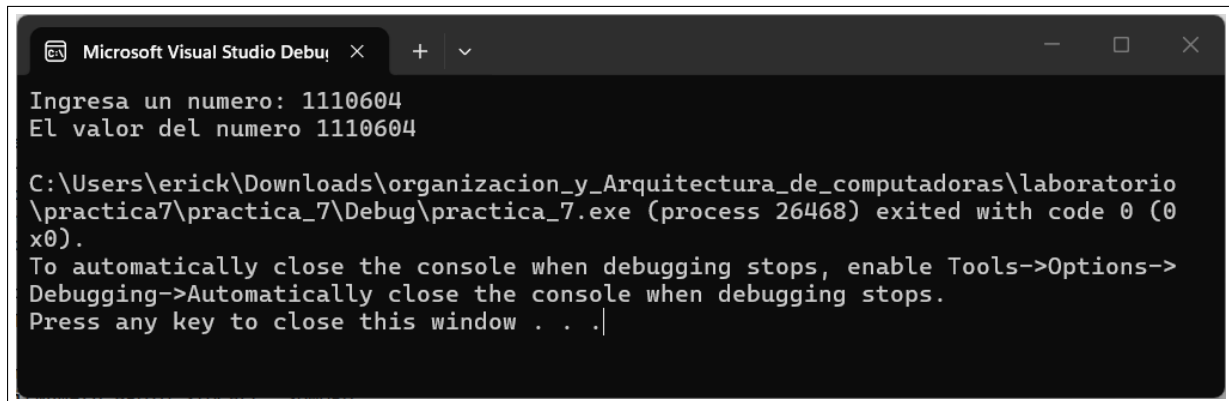
Análisis de instrucción

Instrucción

```
1  pop dx
2  mov byte ptr [bufferTemp], dl
3  ...
4  dec ecx
5  jg _loop_imprimir
```

Descripción Desapila cada dígito en el orden correcto y lo imprime, repitiendo tantas veces como dígitos se contaron.

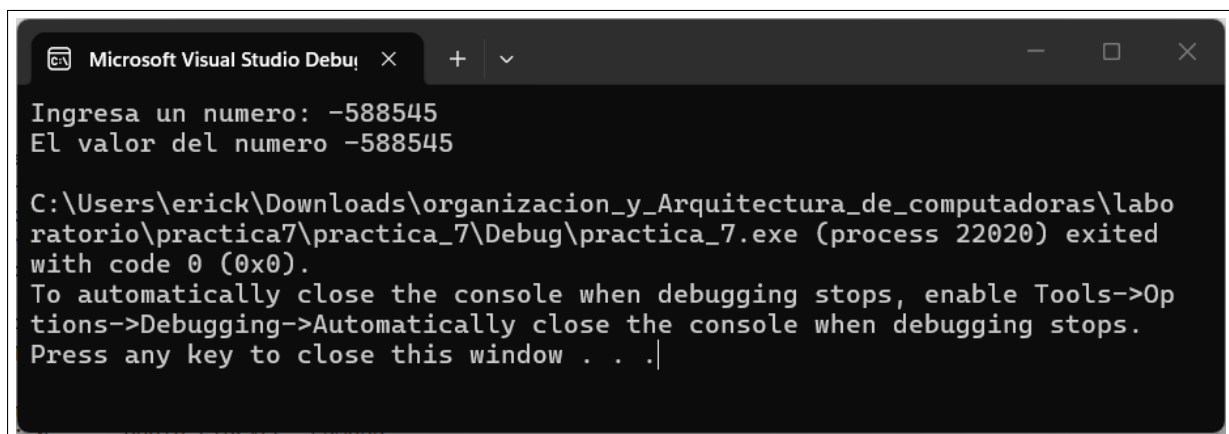
LIFO de la pila resuelve la inversión, los dígitos se apilaron del último al primero, así que al desapilarlos con `POP` salen del primero al último, listos para imprimirse de izquierda a derecha. El contador `ECX`, que registró cuántos dígitos se apilaron, controla el número de iteraciones del bucle de impresión y se preserva con `PUSH/POP` alrededor de la llamada a `WriteConsoleA`, pues esta podría alterarlo.



```
Microsoft Visual Studio Debug Console
Ingresa un numero: 1110604
El valor del numero 1110604

C:\Users\erick\Downloads\organizacion_y_Arquitectura_de_computadoras\laboratorio\practica7\practica_7\Debug\practica_7.exe (process 26468) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figura 3: Lectura de un número con LeerNumero y su impresión con ImprimirNumero.



```
Microsoft Visual Studio Debug Console
Ingresa un numero: -588545
El valor del numero -588545

C:\Users\erick\Downloads\organizacion_y_Arquitectura_de_computadoras\laboratorio\practica7\practica_7\Debug\practica_7.exe (process 22020) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Figura 4: Conversión correcta de un valor negativo, mostrando el signo y los dígitos.

6.4. Resumen de las funciones implementadas

La Tabla 1 sintetiza el propósito de cada procedimiento, la función de la API en que se apoya y el concepto de ensamblador que ejercita.

Tabla 1: Resumen de las funciones de entrada/salida desarrolladas.

Función	Propósito	API usada	Concepto clave
ImprimirCadena	Imprime una cadena de cualquier longitud	WriteConsoleA	Cálculo de longitud (<i>strlen</i>)
LeerCadena	Captura texto y elimina el salto de línea	ReadConsoleA	Limpieza de buffer
LeerNumero	Convierte texto a entero con signo	ReadConsoleA	Esquema de Horner (<i>atoi</i>)
ImprimirNumero	Convierte un entero con signo a texto	WriteConsoleA	Divisiones y pila (<i>itoa</i>)

7. Conclusiones

Realizar esta práctica permitió comprender que incluso las operaciones de entrada y salida más elementales deben construirse a partir de los servicios del sistema operativo y de la manipulación directa de la memoria. Obtener los *handles* con `GetStdHandle` y preparar la pila antes de cada `CALL` dejó en claro el contraste con un lenguaje de alto nivel, donde una sola línea basta para imprimir o leer, frente al ensamblador, donde cada parámetro debe colocarse de forma explícita respetando la convención `stdcall`.

Calcular la longitud recorriendo byte a byte hasta el terminador y limpiar manualmente la secuencia `0Dh 0Ah` de la tecla *Enter* mostró todo lo que un lenguaje de alto nivel resuelve de manera implícita al leer una línea de texto. Comprender la representación de las cadenas a nivel de bytes fue la base para que las funciones operasen sobre datos de longitud arbitraria sin tener que declararla.

La conversión entre texto y enteros fue la parte más interesante. Implementar el esquema de Horner para leer un número y, sobre todo, aprovechar la naturaleza LIFO de la pila para invertir el orden de los dígitos al imprimirlo, evidenció cómo una estructura de datos sencilla resuelve con elegancia un problema concreto.

Referencias

- [1] K. R. Irvine, *Assembly Language for x86 Processors*, 8th. Hoboken, NJ: Pearson, 2020, ISBN: 978-0-13-538165-6.
- [2] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th. Hoboken, NJ: Pearson Education, Inc., 2019, ISBN: 978-0-13-499719-3.
- [3] Microsoft Corporation, *Windows Console API: WriteConsole, ReadConsole and GetStdHandle Functions*, <https://learn.microsoft.com/en-us/windows/console/console-functions>, Microsoft Learn, 2023.
- [4] K. R. Irvine, *Assembly Language for x86 Processors*, 6th. Upper Saddle River, NJ: Prentice Hall, 2011.
- [5] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, ARM. Morgan Kaufmann, 2016.
- [6] A. S. Tanenbaum y T. Austin, *Structured Computer Organization*, 6th. Pearson, 2013.
- [7] A. Elahi, *Computer Systems: Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language*, 2nd. Cham, Switzerland: Springer Nature Switzerland AG, 2022, ISBN: 978-3-030-93448-4. DOI: [10.1007/978-3-030-93449-1](https://doi.org/10.1007/978-3-030-93449-1).
- [8] J. L. Peterson, *Computer Organization and Assembly Language Programming*, Digital Edition. James L. Peterson, 2018.

A. Repositorio de código

El código completo de los tres ejercicios en **MASM x86** desarrollados durante esta práctica se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/ApoloEM/OyAdeC/tree/main/practica7>